

Assignment Project: Task Management App

Overview

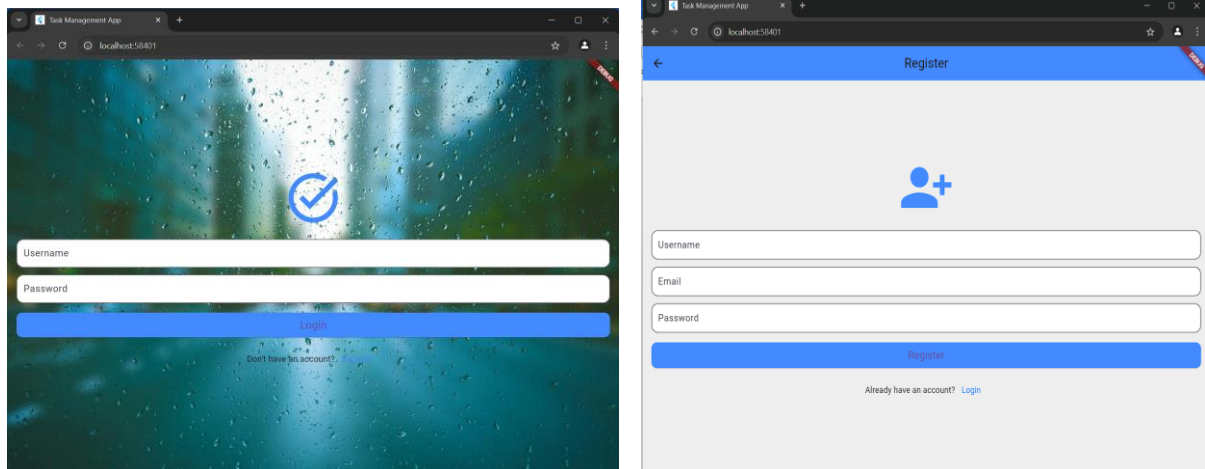
The project aims to develop a simple task management application called **QuickTask**. The app allows users to create, view, edit, and delete tasks. The application was built using the **Flutter** framework for the frontend and **Back4App** for backend services to handle user authentication and task management. This document outlines the core features, functionalities, and a brief walkthrough of the development process.

Features Implemented

1. User Authentication

- Users can create accounts (sign up) and securely log in to the app.
- Each user has their own task dashboard, where they can manage tasks privately.

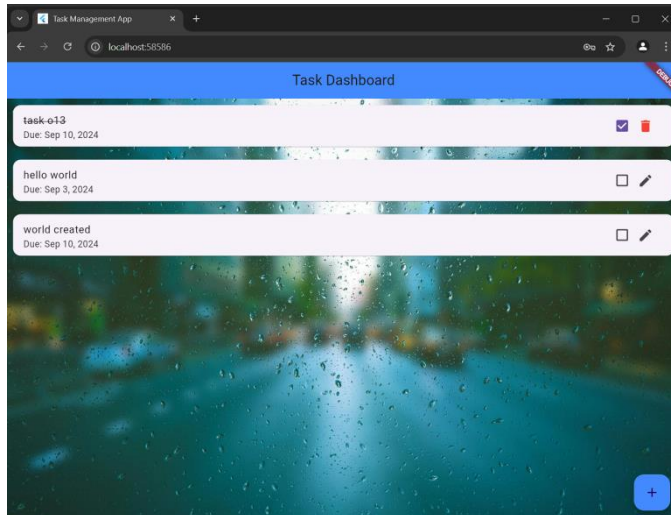
Screenshot: Showing the **Login screen** and **Sign-up screen**.



2. Task Management

- Users can create tasks by adding a title and selecting a due date.
- Tasks are displayed on the task dashboard, and users can manage tasks individually.

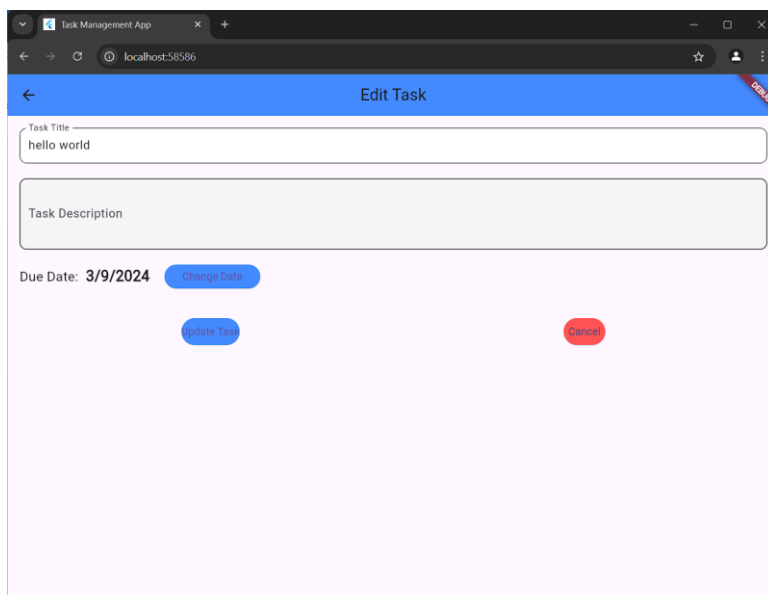
Screenshot: Showing the **Task Dashboard** with some tasks displayed.



3. Task Editing

- Users can edit existing tasks, including changing the title or the due date.
- An edit button is displayed alongside each task for easy access.

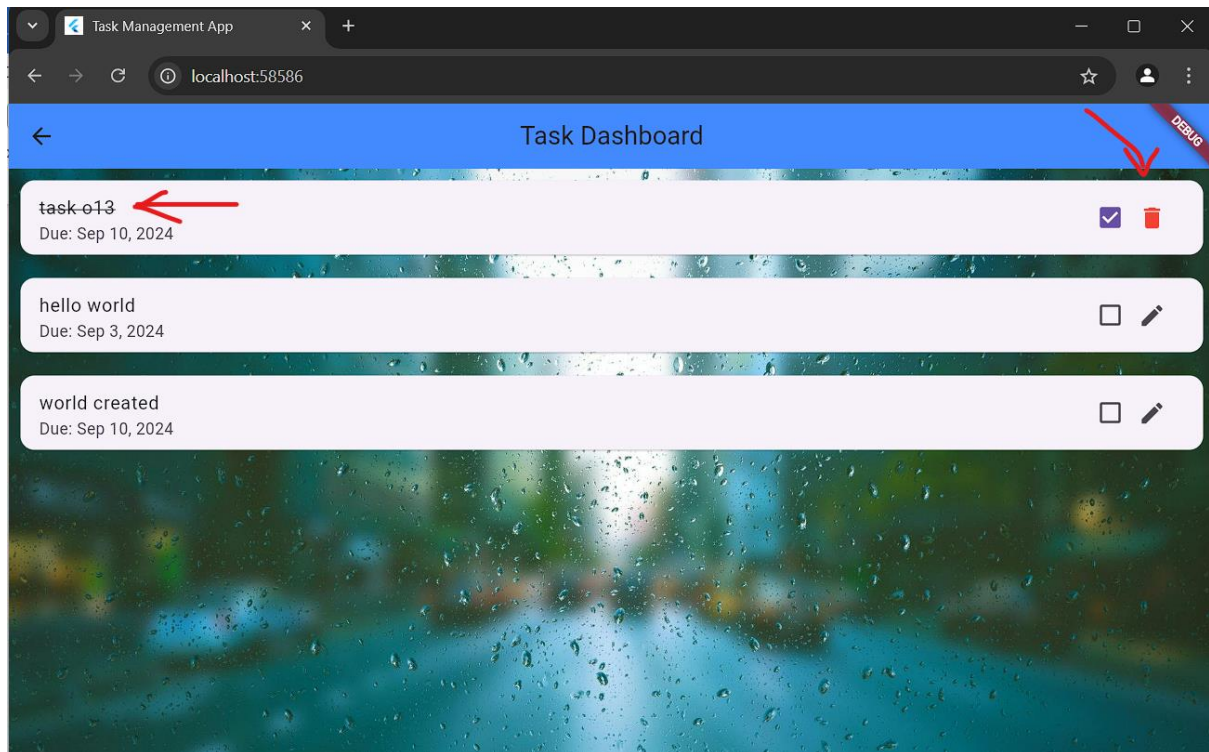
Screenshot: Showing the **Edit Task screen** with task fields being modified.



4. Task Completion and Deletion

- Users can mark tasks as completed.
- Once a task is marked as complete, users are given the option to delete it.

Screenshot: Showing a **completed task** with the **delete button** visible.



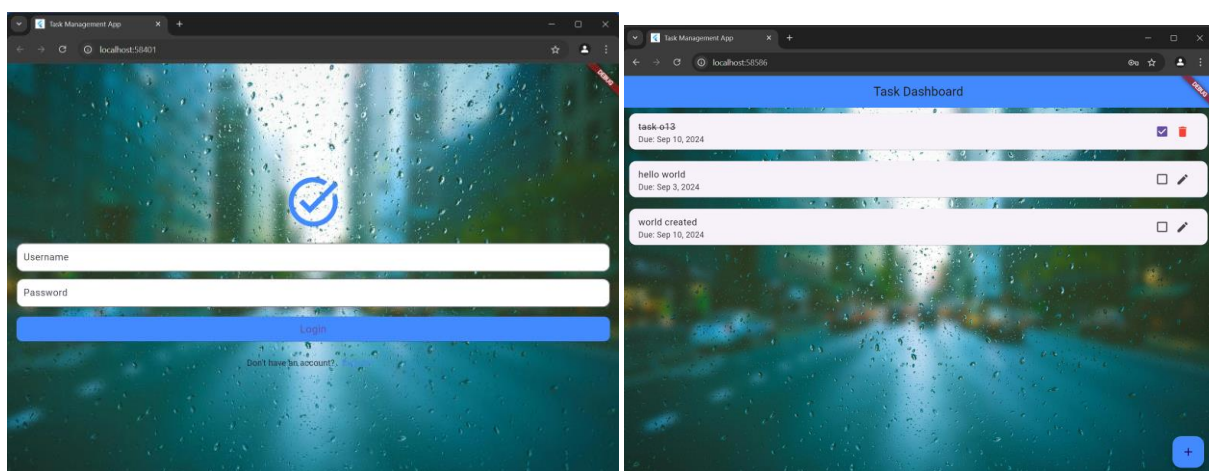
5. User-Specific Task Dashboard

- The app ensures that each user sees only the tasks they have created after logging in.
- Task data is filtered based on the logged-in user, ensuring that tasks are private.

6. UI/UX Enhancements

- The app features an intuitive and simple user interface.
- Background images have been used on login and task dashboard screens to improve the visual experience.

Screenshot: Showing the **Login screen with background image** and **Task Dashboard with background.**



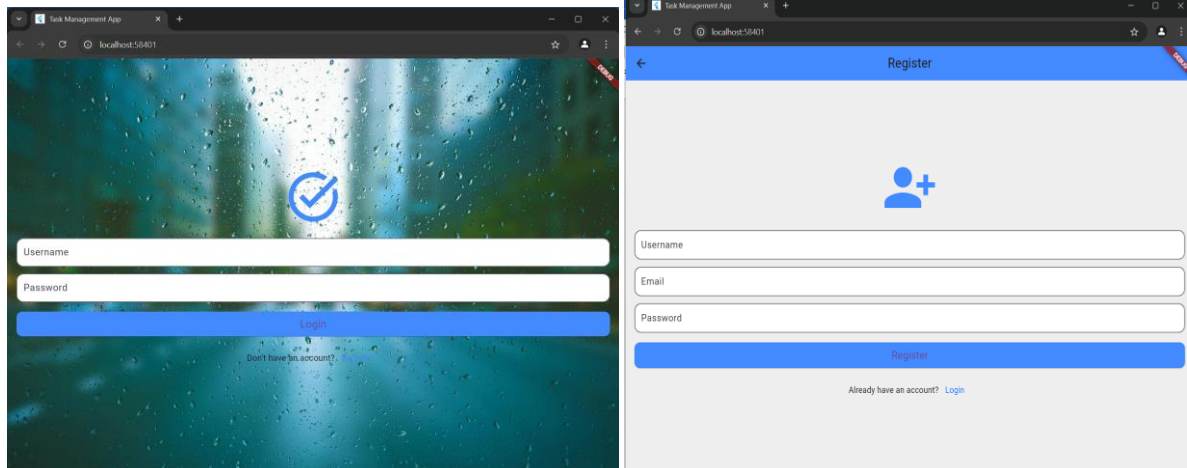
User Flow

1. Sign-Up/Login

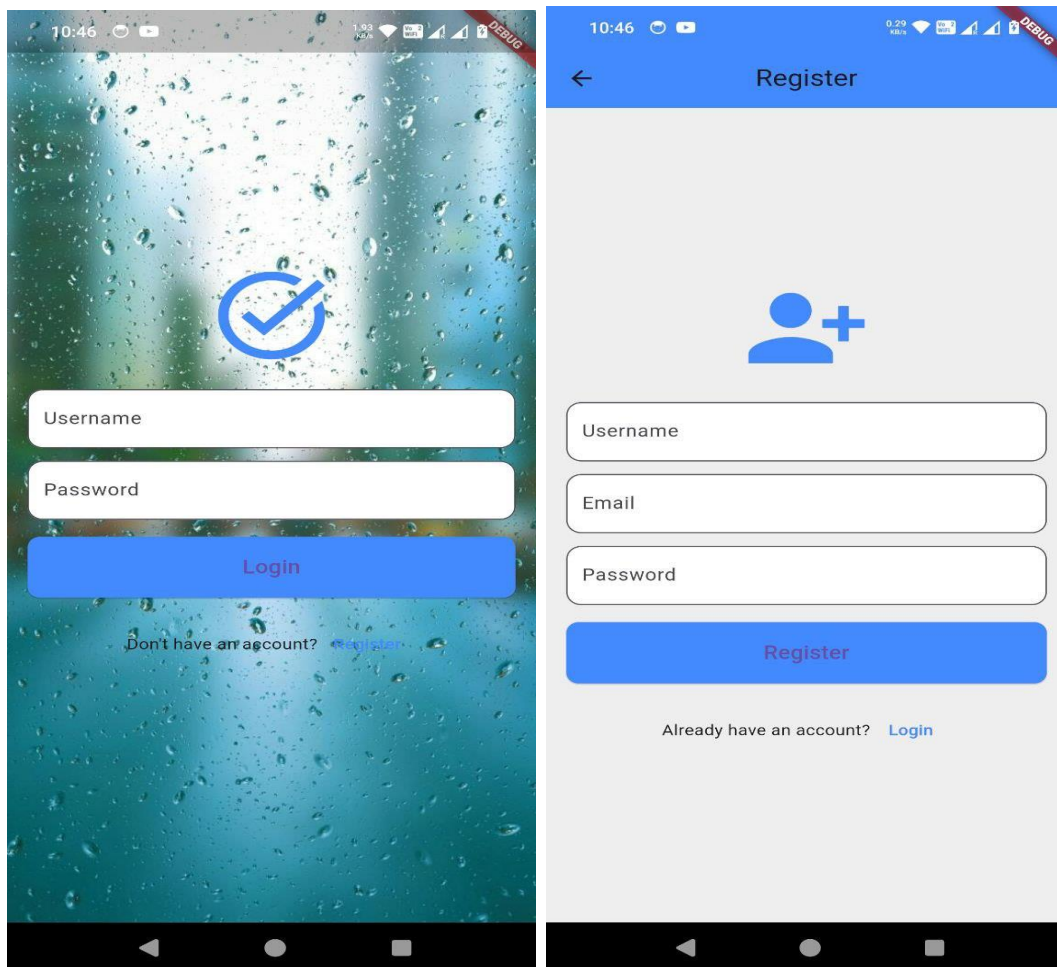
- When a user opens the app, they are greeted with a login screen. If the user doesn't have an account, they can sign up using the "Register" button.

Screenshot: Displaying **Sign-up screen** and the **Login screen** showing both web and Android views.

WEB:



Andorid:

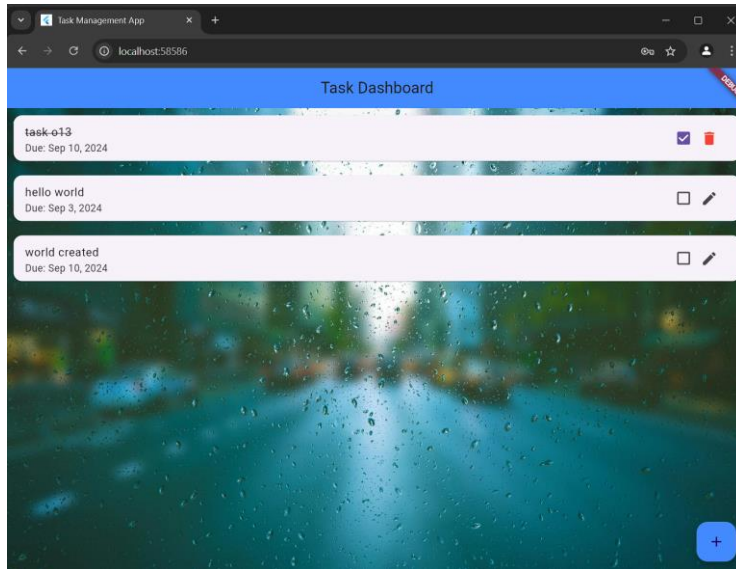


2. Task Dashboard

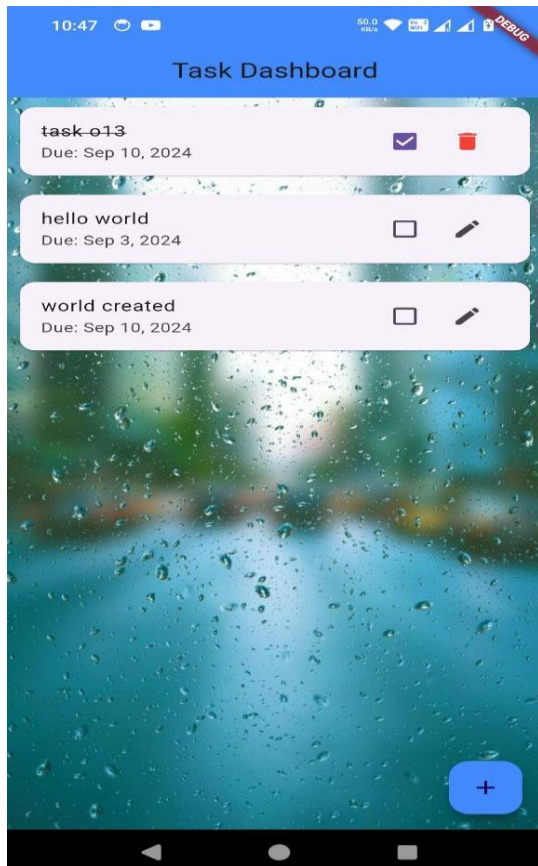
- Upon logging in, users are directed to their task dashboard where they can view their tasks. The dashboard displays tasks with options to mark them as complete, edit, or delete them.

Screenshot: Showing the **Task Dashboard** on both **Web and Android**, showing task list and options.

WEB:



Android:

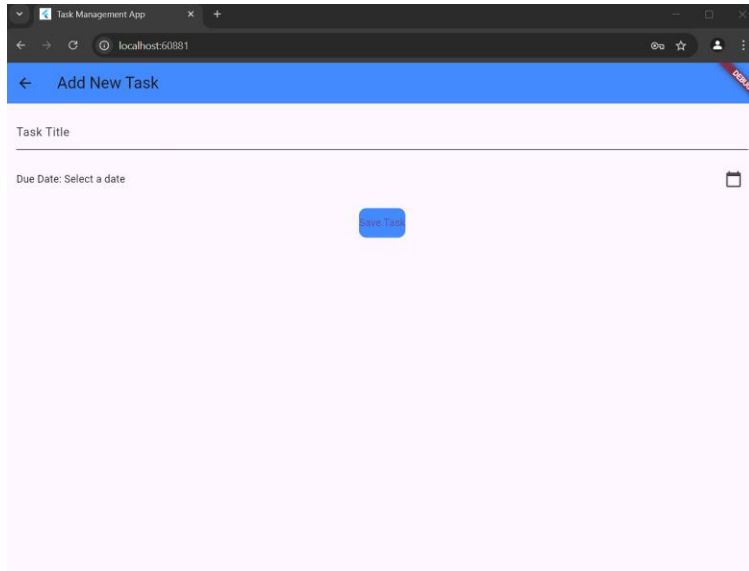


3. Adding a New Task

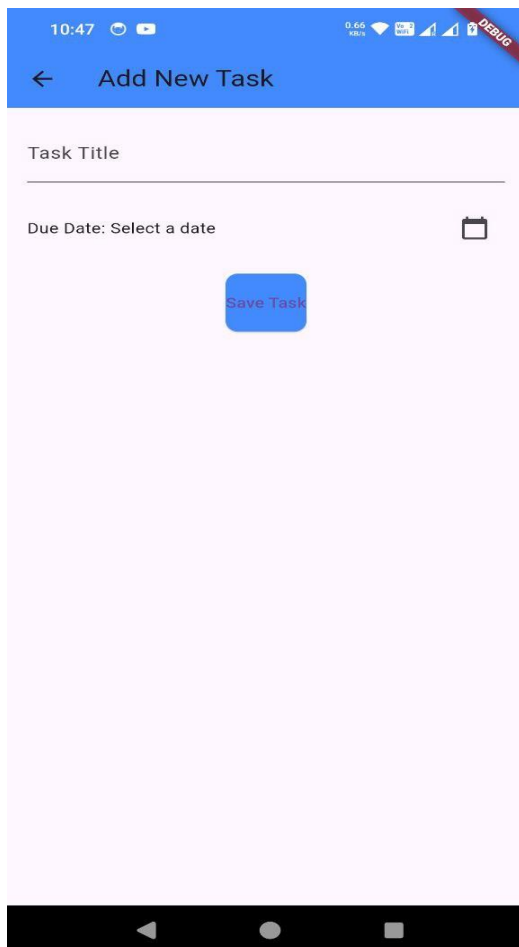
- A floating action button allows users to create a new task. The user can add a task title, set a due date, and save the task.

Screenshot: Showing the **Add Task screen** on Web and Android.

WEB:



Android:

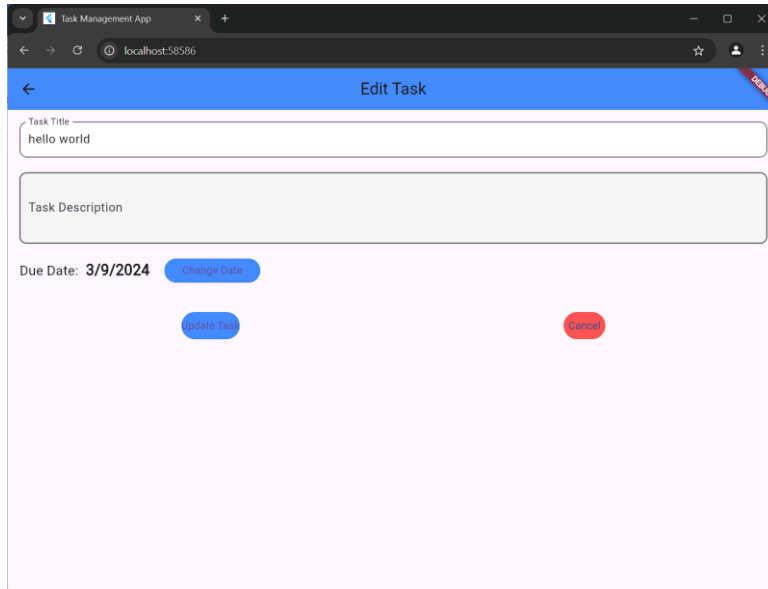


4. Editing and Completing Tasks

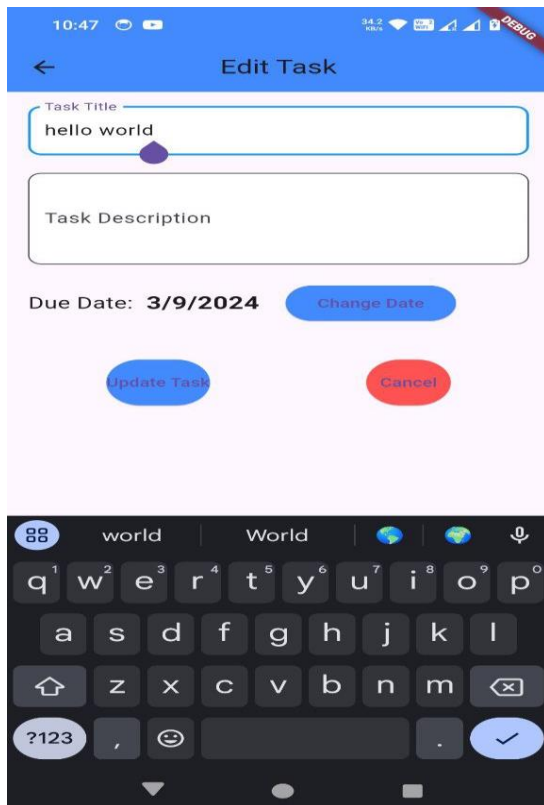
- Users can tap the edit icon next to a task to change the title or due date.
- To mark a task as completed, the user can check the checkbox, which will strike through the task title.

Screenshot: Showing the **Edit Task screen** on both Web and Android.

WEB:



Android:

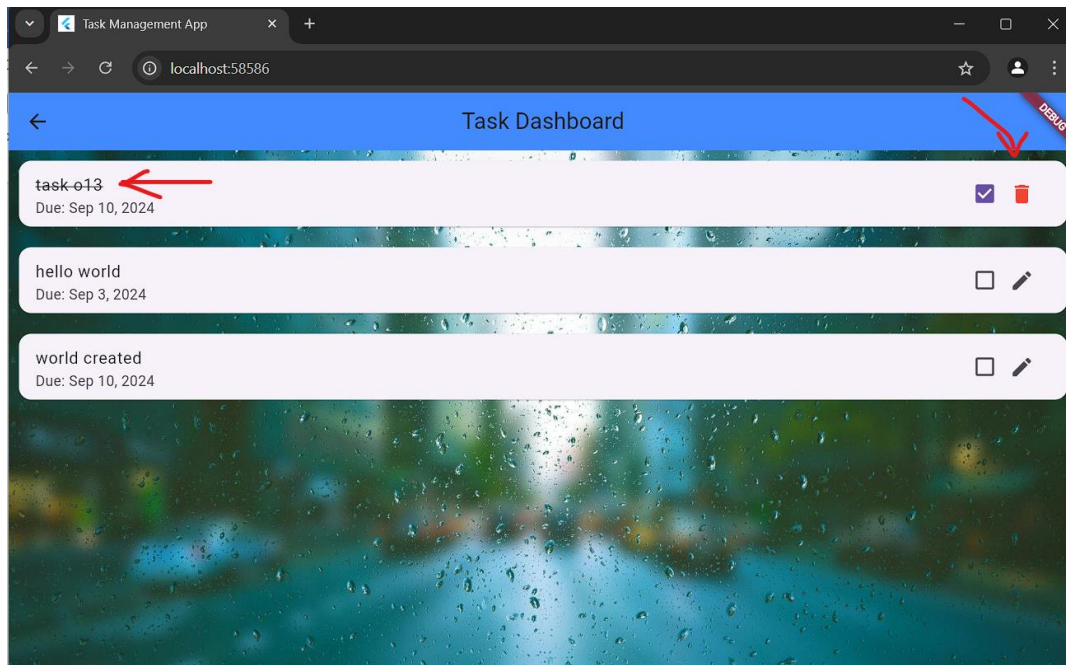


5. Deleting Completed Tasks

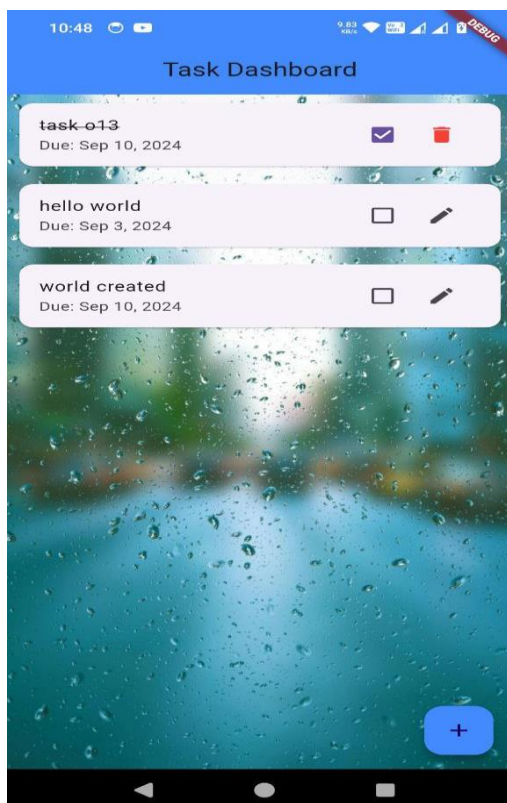
- After marking a task as complete, the delete icon becomes visible. Users can delete completed tasks by tapping this icon.

Screenshot: Display completed tasks with delete options.

WEB:



Android:



Technical Breakdown

- **Frontend:** The frontend was developed using the Flutter framework, which allows for creating cross-platform applications (iOS, Android, Web) with a single codebase. Various Flutter widgets such as TextField, ListView, FloatingActionButton, and FutureBuilder were used to build the user interface.
- **Backend:** Back4App, a cloud-based backend service powered by Parse, was used to handle user authentication and task storage. Back4App offers features like user management and data persistence, enabling easy integration of the backend without managing custom servers.
- **Data Management:** Tasks and user data are stored in the Parse database. Tasks are associated with the users who create them, ensuring that users only see their own tasks.

Development Process

1. Setting Up the Flutter Project

- A Flutter project was initialized with dependencies for Parse Server SDK to handle backend interactions and basic Flutter packages to manage the UI components.

Screenshot: Show the **initial Flutter setup** or **terminal setup** with dependencies listed.

```
lib > screens > Task_board > task_dashboard.dart > TaskDashboardState > build
14 class TaskDashboardState extends State<TaskDashboard> {
PROBLEMS 18 OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
PS D:\Document picture desktop\Bits\assignment\cross_platform_assignment\Task_Management_app\Task_management_app_CPAD_Assignment>
flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[V] Flutter (Channel stable, 3.24.3, on Microsoft Windows [Version 10.0.22631.4249], locale en-IN)
[V] Windows Version (Installed version of Windows is version 10 or higher)
[V] Android toolchain - develop for Android devices (Android SDK version 34.0.0)
[V] Chrome - develop for the web
[X] Visual Studio - develop Windows apps
X Visual Studio not installed; this is necessary to develop Windows apps.
  Download at https://visualstudio.microsoft.com/downloads/.
  Please install the "Desktop development with C++" workload, including all of its default components
[V] Android Studio (version 2024.1)
[V] VS Code (version 1.93.1)
[V] Connected device (3 available)
[V] Network resources

! Doctor found issues in 1 category.
PS D:\Document picture desktop\Bits\assignment\cross_platform_assignment\Task_Management_app\Task_management_app_CPAD_Assignment>
flutter run
Connected devices:
Windows (desktop) • windows • windows-x64 • Microsoft Windows [Version 10.0.22631.4249]
Chrome (web) • chrome • web-javascript • Google Chrome 129.0.6668.68
Edge (web) • edge • web-javascript • Microsoft Edge 129.0.2792.65
[1]: Windows (windows)
[2]: Chrome (chrome)
[3]: Edge (edge)
Please choose one (or "q" to quit): 2
Launching lib/main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome...
```

2. User Authentication Integration

- A login and registration system was developed using Parse's user authentication features. Users are required to log in before accessing the task dashboard.

Screenshot: Display the **user login process** with successful authentication.

WE can see here in back4app database we have user mail id dude@1234: <- we use this .

Name- Nilesh Kumar Mahato Bitsid -2021tm70864
Assignment_Cross_Platform_Application_Development

back4app AI Agent My Apps Dashboard Docs Support Hello, bcalearn990 New app

QuickTaskApp Database Browser 4 objects

		authData Object *	username String *	createdAt Date	password String *	email String	Add a new column
Database	at 1...	(undefined)	DemoHelloWorld	28 Sept 2024 at 1...	(hidden)	b.dhanush@gmail.c...	
Browser	at 0...	(undefined)	Nick2	28 Sept 2024 at 0...	(hidden)	Nick1@1234	
_Role	0	(undefined)	Nick	27 Sept 2024 at 0...	(hidden)	Nick@1234	
_Session	15	(undefined)					
_User	4	(undefined)	nothing	25 Sept 2024 at 1...	(hidden)	dude@1234	
Task	11						

Index Manager

Task Management App localhost:61297

debug

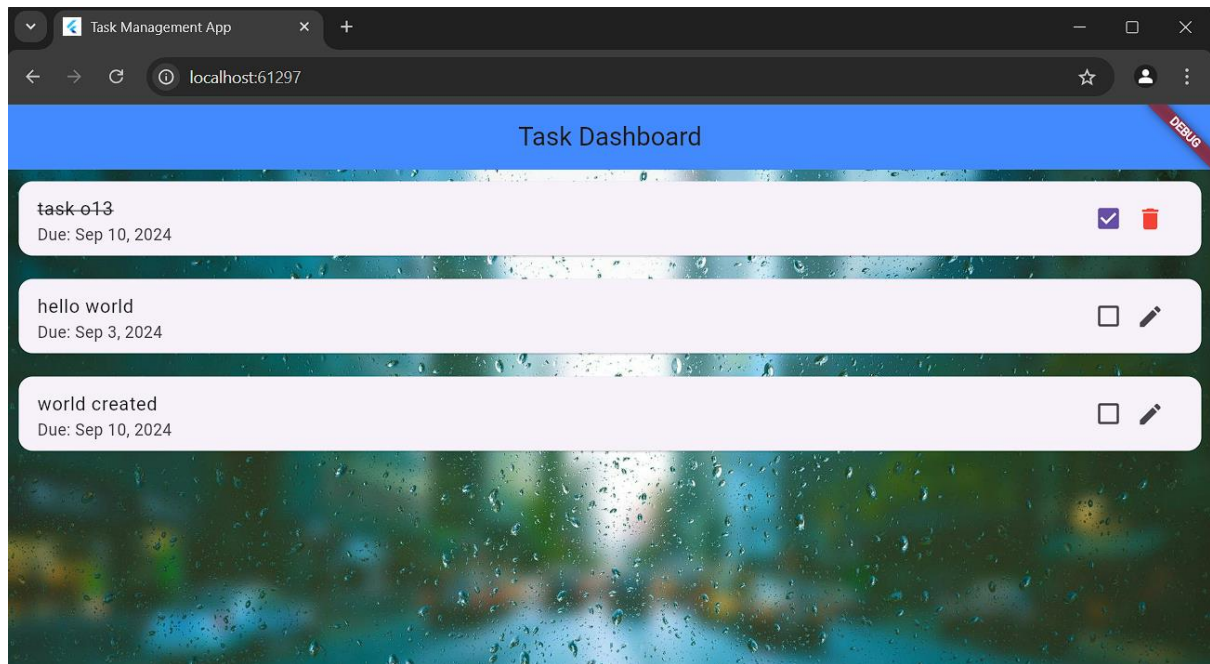
✓

Username
dude@1234

Password
.....

Login

Don't have an account? [Sign up](#)



3. Task Creation, Editing, and Deletion

- The core functionality of task creation, editing, and deletion was implemented using Parse's data storage system. Each task is linked to a specific user, ensuring tasks are private to each account.

Showing the **task creation screen**, **edit screen**, and **delete confirmation**. In video demo..

4. User-Specific Task Display

- After logging in, users can see only their tasks. The app queries Parse to fetch tasks created by the logged-in user, ensuring a personalized experience.

Displaying a user's private **task dashboard**.in video demo

5. UI and User Experience

- The user interface was enhanced with background images and clean layouts for task management. Tasks are displayed in a list format with editing, completion, and deletion options.

Showing the **login screen with background** and **task dashboard with background**. In video demo..

Challenges Faced

1. **User-specific Task Filtering:** One of the main challenges was ensuring that each user sees only their own tasks. This was solved by linking each task to the specific user who created it and querying tasks based on the logged-in user's ID.
 2. **UI Consistency:** Implementing a consistent UI across different screens required careful layout design and testing. The use of Stack to display background images along with task elements required some adjustments to ensure proper alignment.
-

Technologies Used

- **Flutter:** Cross-platform UI toolkit to build the app.
- **Dart:** Programming language for Flutter development.
- **Back4App:** Cloud-based backend for handling user authentication and task management.
- **Parse SDK:** Used to integrate Back4App for user login, sign-up, and task management.
- **Internationalization (intl):** Used to format task due dates.
- **Vscode** – IDE to work on code.

Git code

[https://github.com/nileshkmahato/Task_management_app_CPAD_Assingment]

Demo video:

IN the demo video will:

Display the **user login process** with successful authentication.

Show the **task creation screen**, **edit screen**, and **delete confirmation**.

Displaying a user's private **task dashboard**.

Show the **login screen with background** and **task dashboard with background**.

[<https://youtu.be/rMOT3ledDw4>]